LAPORAN PRAKTIKUM STRUKTUR DATA

QUEUE

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum “Struktur Data Queue (Antrian)” ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan, bimbingan, dan bantuan selama proses praktikum berlangsung.

Laporan ini disusun sebagai salah satu bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Queue (antrian). Dalam laporan ini dibahas konsep dasar queue, prinsip kerjanya, serta penerapannya dalam studi kasus sederhana berupa sistem antrian loket, termasuk operasi dasar seperti enqueue dan dequeue serta fitur tambahan seperti reverse data.

Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat menjadi lebih baik di masa yang akan datang, serta dapat memberikan manfaat dan menambah pemahaman mengenai struktur data queue.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR .....

DAFTAR ISI .....

BAB I PENDAHULUAN .....

1.1 Latar Belakang .....

1.2 Tujuan .....

1.3 Manfaat .....

BAB II PEMBAHASAN

2.1 QueueArray_2511533001& QueuearrayDriver_2511533001 .....

2.2 QueueLinkedList_2511533001 .....

2.3 ReverseData_2511533001 .....

2.4 IterasiQueue_2511533001 .....

2.5 Tugas .....

BAB III KESIMPULAN .....

3.1 Kesimpulan .....

3.2 Saran .....

DAFTAR PUSTAKA .....

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, tidak semua kondisi memungkinkan akses bebas terhadap setiap elemen. Ada situasi tertentu yang memerlukan aturan khusus, di mana data yang pertama dimasukkan akan diproses lebih dulu. Konsep ini dikenal dengan FIFO (First In, First Out) dan diterapkan dalam struktur data Queue atau antrian.

Jika dibandingkan dengan struktur data lain seperti Array atau ArrayList yang dapat diakses secara fleksibel menggunakan indeks, Queue memiliki aturan yang lebih terstruktur karena hanya beroperasi pada dua bagian utama, yaitu bagian depan (front) dan belakang (rear). Cara kerja ini membuat Queue sangat cocok digunakan dalam berbagai kasus seperti sistem antrian loket, pelayanan rumah sakit, serta proses pengolahan data dalam sistem komputer.

Melalui praktikum ini, konsep Queue dipelajari untuk memahami bagaimana data dikelola secara teratur dan sistematis. Selain itu, penerapan Queue dalam studi kasus sederhana juga membantu dalam memahami alur kerja antrian serta operasi dasar yang terjadi di dalamnya seperti enqueue dan dequeue.

1.2 Tujuan

1. Memahami konsep dasar struktur data Queue dan prinsip FIFO (First In, First Out).
2. Mengetahui cara kerja Queue dalam pengelolaan data secara berurutan.
3. Memahami operasi dasar pada Queue seperti enqueue dan dequeue.
4. Menganalisis alur kerja Queue dalam suatu sistem antrian sederhana.
5. Mengembangkan logika pemrograman dalam menyelesaikan permasalahan yang berkaitan dengan antrian.

1.3 Manfaat

1. Meningkatkan pemahaman tentang struktur data Queue dalam pemrograman.
2. Melatih kemampuan berpikir logis dan sistematis dalam mengelola data.
3. Membantu memahami penerapan konsep antrian dalam kehidupan sehari-hari.
4. Memberikan gambaran tentang cara kerja sistem yang menggunakan antrian.
5. Menambah pengalaman dalam menganalisis permasalahan berbasis struktur data.

BAB II

PEMBAHASAN

2.1 QueueArray_2511533001& QueuearrayDriver_2511533001

```java
package pekan4_2511533001;

public class QueueArray_2511533001 {
    int front_3001, rear_3001, size_3001;
    int capacity_3001;
    int array_3001[];

    public QueueArray_2511533001 (int capacity_3001) {
        this.capacity_3001 = capacity_3001;
        front_3001 = this.size_3001 = 0;
        rear_3001 = capacity_3001 - 1;
        array_3001 = new int[this.capacity_3001];
    }

    boolean isFull_3001(QueueArray_2511533001 queue_3001) {
        return (queue_3001.size_3001 == queue_3001.capacity_3001);
    }
    boolean isEmpty_3001(QueueArray_2511533001  queue_3001) {
        return (queue_3001.size_3001 == 0);
    }

    void enqueue_3001 (int item) {
        if (isFull_3001(this))
            return;
        this.rear_3001 = (this.rear_3001 + 1) % this.capacity_3001;
        this.array_3001[this.rear_3001] = item;
        this.size_3001 = this.size_3001 + 1;
        System.out.println(item + "  enqueued to queue");
```

```java
    }

    int dequeue_3001() {
        if (isEmpty_3001(this))
            return Integer.MIN_VALUE;
        int item = this.array_3001[this.front_3001];
        this.front_3001 =(this.front_3001 + 1) % this.capacity_3001;
        this.size_3001 = this.size_3001 - 1;
        return item;
    }
    int front_3001() {
        if (isEmpty_3001(this))
            return Integer.MIN_VALUE;

        return this.array_3001[this.rear_3001];
    }
    int rear_3001() {
        if (isEmpty_3001(this))
            return Integer.MIN_VALUE;
        return this.array_3001[this.rear_3001];
}
void display_3001() {
    int i;
    if (front_3001 == rear_3001) {
        System.out.printf("\nAntrian Kosong\n");
        return;
    }
    for (i = front_3001; i < rear_3001; i++) {
```

```java
        System.out.printf(" %d <--", array_3001[i]);
    }
    return;
    }
}
```

```java
package pekan4_2511533001;

public class QueueArrayDriver_211533001 {

    public static void main(String[] args) {
        QueueArray_2511533001 queue_3001 = new QueueArray_2511533001(1000);
        queue_3001.enqueue_3001(10);
        queue_3001.enqueue_3001(20);
        queue_3001.enqueue_3001(30);
        queue_3001.enqueue_3001(40);
        System.out.println("Item di depan "+ queue_3001.front_3001());
        System.out.println("Item dibelakang " +  queue_3001.rear_3001());
        System.out.println("tampilan queue");
        queue_3001.display_3001();
        System.out.println();
        System.out.println(queue_3001.dequeue_3001() + " dihapus dari queue");
        System.out.println("Item di depan "+ queue_3001.front_3001());
        System.out.println("Item dibelakang " +  queue_3001.rear_3001());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue_3001.display_3001();
    }

}
```

Pembahasan:

1. QueueArray_2511533001 queue_3001 = new QueueArray_2511533001(1000); → Pada driver dibuat objek queue dengan kapasitas 1000, sehingga queue masih dalam kondisi kosong (size = 0).
2. enqueue_3001(10), enqueue_3001(20), enqueue_3001(30), enqueue_3001(40); → Data dimasukkan ke dalam queue secara berurutan.
3. Setiap pemanggilan enqueue_3001() → Program akan mengecek apakah queue penuh menggunakan isFull_3001(). Jika tidak penuh, maka posisi rear digeser dengan (rear + 1) % capacity, lalu data dimasukkan ke array dan size bertambah.
4. Setelah data 10, 20, 30, 40 dimasukkan → Urutan queue menjadi: 10 (depan), diikuti 20, 30, dan 40 (belakang).
5. System.out.println("Item di depan "+ queue_3001.front_3001()); → Menampilkan elemen paling depan queue.
6. System.out.println("Item dibelakang " + queue_3001.rear_3001()); → Menampilkan elemen paling belakang queue.
7. queue_3001.display_3001(); → Menampilkan seluruh isi queue dari depan ke belakang.
8. queue_3001.dequeue_3001(); → Method ini digunakan untuk mengambil sekaligus menghapus elemen dari depan.
9. Sebelum menghapus, dilakukan pengecekan isEmpty_3001() → Untuk memastikan queue tidak kosong.
10. Data pertama yaitu 10 diambil dari posisi front, kemudian front digeser dan size dikurangi → Sehingga data 10 keluar dari queue.
11. Setelah dilakukan dequeue → Urutan queue menjadi: 20 (depan), diikuti 30 dan 40 (belakang).
12. Pemanggilan kembali front_3001(), rear_3001(), dan display_3001() → Digunakan untuk menampilkan kondisi queue setelah satu data dihapus.
13. Dari hasil pengujian pada driver → Terlihat bahwa data yang keluar pertama adalah 10, yaitu data yang pertama kali dimasukkan.
14. Hal ini membuktikan bahwa queue bekerja sesuai prinsip FIFO (First In, First Out), yaitu data yang masuk terlebih dahulu akan keluar terlebih dahulu.Output:

Output:

```
10  enqueued to queue
20  enqueued to queue
30  enqueued to queue
40  enqueued to queue
Item di depan 40
Item dibelakang 40
tampilan queue
 10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan 40
Item dibelakang 40
tampilan queue setelah satu data dihapus
```

2.2 QueueLinkedList_2511533001

```java
package pekan4_2511533001;

import java.util.Queue;
public class QueueLinkedList_2511533001 {

    public static void main(String[] args) {
        Queue<Integer> q_3001 = new LinkedList<>();
        // tambah elemen {0, 1, 2, 3, 4,5} ke antrian
        for (int i = 0; i < 6; i++)
            q_3001.add(i);
        //Menampilkan isi antrian.
        System.out.println("Elemen Antrian " + q_3001);
        //Untuk menghapus kepala antrian.
        int hapus = q_3001.remove();
        System.out.println("Hapus elemen = " + hapus);
        System.out.println(q_3001);
        //Untuk melihat antrian terdepan
        int depan = q_3001.peek();
        System.out.println("Kepala Antrian = " + depan);

        int banyak = q_3001.size();
        System.out.println("Size Antrian = " + banyak);
    }
}
```

Pembahasan:

1. Queue<Integer> q_3001 = new LinkedList<>(); → Membuat queue menggunakan LinkedList sehingga tidak perlu mengatur struktur secara manual.
2. for (int i = 0; i < 6; i++) q_3001.add(i); → Menambahkan data 0 sampai 5 ke dalam queue secara berurutan.
3. add() → Data otomatis masuk ke bagian belakang antrian.
4. Setelah ditambahkan → Urutan queue menjadi 0, 1, 2, 3, 4, 5.
5. System.out.println(q_3001); → Menampilkan isi queue.
6. q_3001.remove(); → Menghapus elemen paling depan, yaitu 0.
7. Setelah dihapus → Queue menjadi 1, 2, 3, 4, 5.
8. q_3001.peek(); → Melihat elemen paling depan tanpa menghapusnya, yaitu 1.
9. q_3001.size(); → Menampilkan jumlah elemen, yaitu 5.
10. Dari hasil tersebut → Queue bekerja sesuai prinsip FIFO (First In, First Out).

Output:

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

2.3 ReverseData_2511533001

```java
package pekan4_2511533001;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533001 {

    public static void main(String[] args) {
        Queue<Integer> q_3001 = new LinkedList<Integer>();
        q_3001.add(1);
        q_3001.add(2);
        q_3001.add(3);
        System.out.println("Sebelum reverse" + q_3001);
        Stack<Integer> s_3001 = new Stack<Integer>();
        while (!q_3001.isEmpty()) { // Q -> S
            s_3001.push(q_3001.remove());
        }
        while (!s_3001.isEmpty()) { // S -> Q
            q_3001.add(s_3001.pop());
    }
        System.out.println("Sesudah reverse= " + q_3001); // [3, 2, 1]
}
}
```

1. Queue<Integer> q_3001 = new LinkedList<>(); → Membuat queue menggunakan LinkedList sebagai penyimpanan data.

2. q_3001.add(1), add(2), add(3); → Menambahkan data ke dalam queue secara berurutan sehingga isi awalnya adalah 1, 2, 3.

3. System.out.println("Sebelum reverse" + q_3001); → Menampilkan isi queue sebelum dilakukan proses pembalikan.

4. Stack<Integer> s_3001 = new Stack<>(); → Membuat stack sebagai struktur bantu untuk proses reverse.

5. while (!q_3001.isEmpty()) → Semua elemen queue dipindahkan ke stack menggunakan remove() dan push() (Q → S).

6. Proses tersebut membuat urutan di stack menjadi terbalik karena sifat stack adalah LIFO.

7. while (!s_3001.isEmpty()) → Elemen dari stack dikembalikan lagi ke queue menggunakan pop() dan add() (S → Q).

8. System.out.println("Sesudah reverse= " + q_3001); → Menampilkan hasil akhir queue yang sudah terbalik menjadi 3, 2, 1.

Output:

```
Sebelum reverse[1, 2, 3]
Sesudah reverse= [3, 2, 1]
```

2.4 IterasiQueue_2511533001

```java
package pekan4_2511533001;

import java.util.*;

public class IterasiQueue_2511533001 {
    public static void main(String[] args)
    {
        Queue<String> q_3001 = new LinkedList<>();

        q_3001.add("Praktikum");
        q_3001.add("Struktur");
        q_3001.add("Data");
        q_3001.add("Dan");
        q_3001.add("Algoritma");
        Iterator<String> iterator_3001 = q_3001.iterator();
        while (iterator_3001.hasNext()) {
            System.out.print(iterator_3001.next() + " ");
        }
    }
}
```

Pembahasan:

1. Queue<String> q_3001 = new LinkedList<>(); → Membuat queue bertipe data String menggunakan LinkedList.
2. q_3001.add(...) → Menambahkan beberapa data ke dalam queue yaitu "Praktikum", "Struktur", "Data", "Dan", "Algoritma".
3. Setelah ditambahkan → Urutan queue menjadi: Praktikum, Struktur, Data, Dan, Algoritma.
4. Iterator<String> iterator_3001 = q_3001.iterator(); → Membuat iterator untuk melakukan penelusuran (iterasi) terhadap elemen queue.
5. while (iterator_3001.hasNext()) → Perulangan dilakukan selama masih ada elemen berikutnya di dalam queue.
6. iterator_3001.next() → Mengambil elemen satu per satu secara berurutan dari depan ke belakang.
7. System.out.print(...) → Menampilkan setiap elemen queue dalam satu baris.
8. Dari proses iterasi → Seluruh elemen queue dapat ditampilkan tanpa menghapus data yang ada di dalamnya.

```
Praktikum Struktur Data Dan Algoritma
```

2.5 Tugas

1. Queue_25115330001

```
package pekan4_2511533001;
    public class Queue_25115330001 {
        int front_3001, rear_3001, max_3001;
        String queue_3001[];

        public Queue_25115330001(int max_3001) {
            this.max_3001 = max_3001;
            queue_3001 = new String[max_3001];
            front_3001 = -1;
            rear_3001 = -1;
        }

        public boolean isEmpty_3001() {
            return (front_3001 == -1);
        }

        public boolean isFull_3001() {
            return (rear_3001 == max_3001 - 1);
        }

        public void enqueue_3001(String data_3001) {
            if (isFull_3001()) {
                System.out.println("Antrian penuh");
            } else {
                if (isEmpty_3001()) {
                    front_3001 = 0;
                }
                rear_3001++;
                queue_3001[rear_3001] = data_3001;
                System.out.println("Data berhasil ditambahkan ke antrian");
            }
```

```java
        }

        public void dequeue_3001() {
            if (isEmpty_3001()) {
                System.out.println("Antrian kosong");
            } else {
                System.out.println(queue_3001[front_3001] + " telah dilayani");
                if (front_3001 == rear_3001) {
                    front_3001 = rear_3001 = -1;
                } else {
                    front_3001++;
                }
            }
        }

        public void display_3001() {
            if (isEmpty_3001()) {
                System.out.println("Antrian kosong");
            } else {
                System.out.println("Isi antrian:");
                int no= 1;
                for (int i = front_3001; i <= rear_3001; i++) {
                    System.out.println(no + ". " + queue_3001[i]);
                    no++;
                }
            }
        }

        public void reverse_3001() {
```

```java
            if (isEmpty_3001()) {
                System.out.println("Antrian kosong!");
            } else {
                int start = front_3001;
                int end = rear_3001;
                while (start < end) {
                    String temp = queue_3001[start];
                    queue_3001[start] = queue_3001[end];
                    queue_3001[end] = temp;
                    start++;
                    end--;
                }
            }
        }
    }
```

Pembahasan:

1.int front_3001, rear_3001, max_3001; String queue_3001[]; → Mendeklarasikan variabel queue dan array sebagai tempat penyimpanan data.

2. Queue_25115330001(int max_3001) → Constructor untuk menentukan kapasitas antrian.

3. queue_3001 = new String[max_3001]; → Membuat array sesuai kapasitas yang ditentukan.

4. front_3001 = -1; rear_3001 = -1; → Menandakan bahwa antrian awalnya kosong.

5. isEmpty_3001() → Mengecek apakah antrian kosong (front = -1).

6. isFull_3001() → Mengecek apakah antrian sudah penuh (rear = max - 1).

7. enqueue_3001(String data_3001) → Method untuk menambahkan data ke antrian.

8. Jika antrian kosong → front di-set ke 0, lalu rear bertambah dan data dimasukkan ke array.

9. dequeue_3001() → Method untuk menghapus data dari bagian depan antrian.

10. Jika hanya satu data → front dan rear dikembalikan ke -1, jika tidak maka front digeser.

11. display_3001() → Menampilkan seluruh isi antrian dari depan ke belakang.

12. reverse_3001() → Membalik urutan data dengan menukar elemen dari depan dan belakang.

2.AntrianLoket_2511533001

```java
package pekan4_2511533001;

import java.util.*;

public class AntrianLoket_2511533001 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Queue_25115330001 antrian = new Queue_25115330001(10);

        int pilihan;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input.nextLine();
                    antrian.enqueue_3001(nama);
                    break;

                case 2:
                    antrian.dequeue_3001();
                    break;

                case 3:
                    antrian.display_3001();
                    break;

                case 4:
                    antrian.reverse_3001();
                    antrian.display_3001();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid");
            }
        } while (pilihan != 5);

        input.close();
    }
}
```

Pembahasan:

Queue_2511533001 antrian = new Queue_2511533001(10); → Membuat objek antrian dengan kapasitas maksimal 10 data untuk menyimpan nama pelanggan.

int pilihan; → Variabel untuk menyimpan pilihan menu dari pengguna.

do { ... } while (pilihan != 5); → Perulangan digunakan agar program terus berjalan dan menampilkan menu sampai pengguna memilih keluar.

System.out.println("=== PROGRAM ANTRIAN LOKET ==="); → Menampilkan daftar menu yang dapat dipilih oleh pengguna.

pilihan = input.nextInt(); → Mengambil input pilihan dari pengguna.

switch (pilihan) → Digunakan untuk menentukan aksi berdasarkan pilihan menu yang dipilih.

case 1: → Digunakan untuk menambahkan data ke dalam antrian.

antrian.enqueue_3001(nama); → Memasukkan nama pelanggan ke dalam antrian.

case 2: → Digunakan untuk menghapus data dari antrian.

antrian.dequeue_3001(); → Menghapus data paling depan sesuai prinsip FIFO.

case 3: → Digunakan untuk menampilkan isi antrian.

antrian.display_3001(); → Menampilkan seluruh data yang ada dalam antrian.

case 4: → Digunakan untuk membalik urutan antrian.

antrian.reverse_3001(); → Membalik urutan data dalam antrian, lalu ditampilkan.

case 5: → Digunakan untuk keluar dari program.

System.out.println("Program selesai"); → Menampilkan pesan bahwa program telah selesai.

default: → Menangani input yang tidak sesuai dengan menu.

System.out.println("Pilihan tidak valid!"); → Menampilkan pesan kesalahan jika input salah.

Output:

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Ihsanul
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Zaky
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Ihsanul
2. Zaky

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian:
1. Zaky
2. Ihsanul

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Zaky telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Ihsanul

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
```

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum mengenai Queue, dapat disimpulkan bahwa Queue merupakan struktur data linier yang menggunakan prinsip FIFO (First In, First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Queue dapat direpresentasikan menggunakan beberapa cara, seperti array maupun LinkedList.

Operasi dasar seperti enqueue, dequeue, serta pengecekan kondisi antrian memiliki waktu eksekusi yang cepat (O(1)), sehingga efisien dalam pengelolaan data secara berurutan. Penerapan Queue pada praktikum ini, seperti pada sistem antrian loket, menunjukkan bahwa konsep ini banyak digunakan dalam berbagai situasi yang membutuhkan pengolahan data secara terstruktur. Selain itu, queue juga dapat dikembangkan dengan fitur tambahan seperti reverse data.

Dengan demikian, Queue tidak hanya penting untuk dipahami secara teori, tetapi juga memiliki manfaat dalam penyelesaian permasalahan nyata yang berkaitan dengan pengolahan data secara sistematis.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari dan mempersiapkan diri sebelum pelaksanaan praktikum.

DAFTAR PUSTAKA

[1] Oracle, “Queue Interface (Java Platform SE Documentation),” [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html